



DesignWare® Cores Cryptography Software Library Software Development PKA Plug-in User Guide

DWC Cryptography Software Library SDK - Product Code: B273-0

Copyright Notice and Proprietary Information

© 2023 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Export Control

Some DesignWare Security IP may be restricted for export by government encryption regulations. Government export licenses and/or reporting may be required by law prior to transfer of the technology. A compliance check by authorized personnel is needed prior to transfer, as requirements vary depending on the product and the locations involved.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History	4
Reference Documents	5
1 Introduction	6
1.1 Audience.....	6
1.2 Assumptions	6
1.3 Acronyms	6
2 PKA Plug-in Build Options	8
2.1 Example Command Line Build.....	8
3 DWC CSL Build Options.....	9
3.1 PKA Configuration	9
3.2 Example Command Line Build.....	9
4 Math Plug-in	11
4.1 Definition.....	11
4.2 Member Variables	13
4.3 Operations.....	13
4.3.1 Initialization.....	13
4.3.2 De-initialization.....	14
4.4 Updated Operations	14
4.4.1 RSA Sign.....	14
4.4.2 RSA Verify	15
4.4.3 RSA Encrypt.....	15
4.4.4 RSA Decrypt	15
4.4.5 ECC Sign.....	16
4.4.6 ECC Verify	16
4.4.7 ECC Encrypt	17
4.4.8 ECC Decrypt.....	17
4.4.9 ECC Make Key	18

Revision History

Doc. Revision	Date	Description
6513-001 2.20a	February 2023 August 2020	▪ Removed obsolete sections. ▪ Initial version.

Reference Documents

The following reference documents are used in this application note:

- *DesignWare® Cores Cryptography Software Library Software Development Kit User Guide, v5.00a.*

1 Introduction

DWC Cryptography Software Library features a Plug-in architecture which allows the developer to link specific modules to a set of plug-in structures during run-time. These plug-ins are designed to have a consistent interface allowing the user to perform pure software or hardware accelerated operations with the same set of APIs.

This document specifies how the PKA can be used with the Math Plug-in defined in the DWC Cryptography Software Library. It also describes the Math Plug-in's available set of APIs that contain its main functionalities.

The PKA is being given support in Math Plug-in for math operations such as: (i) RSA Modular Exponentiation; (ii) RSA CRT Modular Exponentiation; (iii) ECC Point Multiplication, and; (iv) ECC Shamir's Trick. Using the Math Plug-in for PKA, you can have these methods hardware accelerated:

- *RSA Sign & Verify*
- *RSA Encrypt & Decrypt*
- *ECC Make Key*
- *ECC Sign & Verify*
- *ECC Encrypt & Decrypt*

Non-Blocking/IRQ Mode:

If the Non-Blocking mode gets enabled, ECC and RSA APIs where the core algorithm is performed may return **ELPEINPROGRESS** to indicate that the operation is not finished yet. The user can enable this mode using the specific APIs for that, which are described in the next Chapters. The user must not enqueue further work to any Math Plug-in related API until the pending API is finished. An API is considered to have finished the job in Non-Blocking Mode whenever the callback gets executed.

1.1 Audience

This guide is intended for software developers that plan to understand how to use the DWC Cryptography Software Library Plug-ins infrastructure for hardware-accelerated operations using PKA.

1.2 Assumptions

This guide assumes that you are familiar with:

- Software programming concepts and programming in the C language, and
- The big picture of your company's product software and how DWC Cryptography Software Library Plug-ins fits into it.

1.3 Acronyms

The following acronyms are used in this document:

Acronym	Definition
API	Application Programming Interface

Acronym	Definition
RSA	Rivest-Shamir-Adleman
ECC	Elliptical Curve Cryptography
PKA	Public Key Accelerator
IRQ	Interrupt Request
IP	Intellectual Property

2 PKA Plug-in Build Options

The new plug-in source code in conjunction with DWC Cryptography Software Library extends the PKA IP device driver into a higher-level of abstraction that allows the user to perform Public Key Cryptographic operations in a couple of basic steps that abstract the user from the hardware configuration parts.

These new extensions allow the user to very easily plug-in the crypto IP's device drivers to the DWC Cryptography Software Library's new plug-in infrastructure, since they were tailored for this purpose – Hardware acceleration of the DWC CSL's cryptographic algorithms.

Table 3-1 shows the new build flags added to *pka* project (PKA device driver). If the user wants to build the PKA driver with the new plug-in source code, then he use the new *plugins* Makefile target. In order to specify the DWC CSL header file path, the symbols shown in Table 3-1 must be defined.

Table 2-1 PKA Plugin Build Options

Symbol	Use
CSL	Define the CSL Header file path. Required by the plugin code since it uses CSL definitions and some APIs.

2.1 Example Command Line Build

The main Makefile was updated in the root directory of the *pka* project – *Makefile*. A new build target – *plugins* – was added for plugin building purposes.

In order to build PKA device driver library with the plugin functions included, the user use the following command line to invoke the above-mentioned Makefile in the project's root directory:



Note

Before running the command-line below, make sure your CSL library *elpsoft_defines.h* header file is populated with your configuration's variables

```
make plugins CC='cc' AR='ar' CFLAGS='custom_flags' ' CSL='path to CSL headers'
```

This generates a *libpka.a* static library file that contains the PKA driver and the plug-in code ready to interface with DWC CSL. This library is linked in conjunction with DWC CSL library in the final application. The procedure explained in Section 4, which demonstrates the new updates on the DWC CSL build system.

3 DWC CSL Build Options

The new Plug-in infrastructure brings the addition of a new set of environment flags to the DWC Cryptography Software Library build system. The overall Plug-in infrastructure is enabled by setting the **ELP_PLUGINS** environment variable. Then, each of the supported Plug-ins can be enabled independently by setting their associated environment variables: (i) Hash Plug-in is **ELP_HASH_PLUGIN**; (ii) Cipher Plug-in is **ELP_CIPHER_PLUGIN**; (iii) Math Plug-in is **ELP_MATH_PLUGIN**, and; (iv) Asymmetric Plug-in is **ELP_ECC_PLUGIN**. More info about their use can be found in Table 4-1.

Table 4-1 CSL Plug-ins Build Options

Symbol	Use
ELP_PLUGINS	General build option that adds the interface of the Plug-in infrastructure to the CSL build.
ELP_HASH_PLUGIN	Combined with ELP_PLUGINS, this build option integrates the Hash Plug-in interface to the CSL build.
ELP_CIPHER_PLUGIN	Combined with ELP_PLUGINS, this build option integrates the Cipher Plug-in interface to the CSL build.
ELP_MATH_PLUGIN	Combined with ELP_PLUGINS, this build option integrates the Math Plug-in additions to the CSL build.
ELP_ECC_PLUGIN	Combined with ELP_PLUGINS, this build option integrates the Asymmetric Plug-in interface to the CSL build.

3.1 PKA Configuration

PKA can be used to accelerate the DWC Cryptography Software Library Public Key algorithms through the updated Math Plug-in infrastructure. By defining **ELP_HW_MATH_PLUGIN** environment variable to the build system, the DWC CSL library is built by including PKA plugin init and de-init APIs into the Math plug-in source file (*plugin_math.c*), if enabled. To build successfully, this requires the *hw_math_plugin.h* header file, so make sure you add the necessary compiler flags to the build system when you are building DWC CSL.

Table 4-2 presents the new build options related to the inclusion of the PKA plug-in in the DWC CSL.

Table 4-2 CSL PKA Build Options

Symbol	Use
ELP_HW_MATH_PLUGIN	Adds the module that interfaces with the hardware as a Math Plug-in to the CSL build. Must be combined with ELP_PLUGINS variable.

3.2 Example Command Line Build

DWC Cryptography Software Library continues to be built as usual. The only difference remain on the environment variable defines used prior to starting the build process. The user has to define at least the following variables:

1. ELP_PLUGINS
2. ELP_MATH_PLUGIN
3. ELP_HW_MATH_PLUGIN

In order to build a DWC CSL library with the PKA plugin support, the user use the following command line to invoke the main Makefile in the project's root directory:

```
make CFLAGS='path to Driver SDK or PKA headers'
```

This generates a *libelpcrypt.a* static library file that contains the CSL APIs that the user has selected to build. This library is linked in conjunction with PKA device driver library in the final application.

When linking the final application make sure you add this line into the linking process if you want to include DWC CSL with PKA Plugin support:

```
-L'path to libraries' -lelpcrypt -lpka -lpdu
```

4 Math Plug-in

Hardware accelerated RSA and ECC operations are implemented through the Math Plug-in schema defined in the *src/headers/elpsoft_math.h* header file in the type defined structure named *elp_math_descriptor*. This structure has been updated to support a new series of function pointers to various function prototypes, which are responsible for the RSA and ECC core algorithms required by the DWC Cryptography Software Library.

4.1 Definition

This plug-in received the following additions to its definition:

```

/** math descriptor */
typedef struct _elp_math_descriptor {

...

#ifdef ELP_MATH_PLUGIN
/** RSA MODEXP Accelerators
*/
int (*rsaa_modexp)(void *instance,
                  const uint8_t *in, uint32_t inlen,
                  uint8_t *out, uint32_t *outlen,
                  elprsa_key *key, int which);
int (*rsaa_modexp_crt)(void *instance,
                      const uint8_t *in, uint32_t inlen,
                      uint8_t *out, uint32_t *outlen,
                      elprsa_key *key, int which);

/** ECC Accelerators
*/
int (*ecca_ptmul)(void *instance,
                 void *k, elpecc_point *G, elpecc_point *R,
                 const elpecc_set_type *type);
int (*ecca_ptmul_out)(void *instance,
                    elpecc_point *R,
                    const elpecc_set_type *type);
int (*ecca_mul2add)(void *instance,
                  elpecc_point *A, void *kA,
                  elpecc_point *B, void *kB,
                  elpecc_point *C,
                  const elpecc_set_type *type);
int (*ecca_mul2add_out)(void *instance,
                      elpecc_point *C,
                      const elpecc_set_type *type);
int (*ecca_ptadd)(void *instance,
                 elpecc_point *A, elpecc_point *B,
                 elpecc_point *C, const elpecc_set_type *type);

/** Generic containers
*/
void *driver_instance;
void *cb_param;
/* irq callback */
void (*cb) (struct _elp_math_descriptor *instance,
            uint32_t info, void *param);

int mode;
int current_op;
elpecc_key *ephkey;
elpecc_key pubkey;
void *sm2_k; // random k sm2 used by sign and encrypt APIs
#endif
} elp_math_descriptor;

```

4.2 Member Variables

The following members are required by the DWC Cryptography Software Library internal functions for the Plug-in infrastructure to function properly. They are:

- *driver_instance*
 - opaque instance used to store proprietary driver information
- *cb_param*
 - opaque instance used to store user information
- *mode*:
 - integer variable used to store the plugin mode (SW or HW)
- *current_op*
 - integer variable used to track the current operation PKA is performing
- *ephkey*
 - ECIES Ephemeral key struct pointer to store state when using non-blocking mode
- *pubkey*
 - ECDSA key struct pointer to store state when using non-blocking mode on a Sign operation
- *sm2_k*
 - SM2 key struct pointer to store state when using non-blocking mode on a Sign operation

4.3 Operations

This section focuses in describing every API related to the new additions to the Math Plug-in.

The following APIs are the recommended interface to make use of the updated Math Plug-in infrastructure. They are divided in different groups:

- *Initialization*
- *De-initialization*

4.3.1 Initialization

The initialization API requires a pointer to the Math plug-in structure to be initialized. This API abstracts the user from all the IP configuration, which gets specified by the *hwip* parameter where you select the hardware IP to use. The Math plug-in structure passed to this API also gets its function pointers initialized to point to the functions responsible for the hardware acceleration of the core RSA and ECC algorithms.

The caller may enable the **Non-Blocking/IRQ** mode by sending a *mp_callback*-type function address in the *cb* parameter. Optionally, the user can also send a generic parameter in the *cb_param* field. This callback is executed after an operation is complete, and the associated *cb_param* field is sent to the callback as a parameter.

```
extern int csl_math_plugin_load(elp_math_descriptor *plugin,
                               void *instance, int mode,
                               csl_mp_callback cb, void *cb_param);
```

Parameters:

- *plugin [in/out]*
 - Pointer to the plugin structure to be initialized

- *instance [in]*
 - Pointer to the driver instance to be used by the plugin
- *mode [in]*
 - Integer variable used to select the desired mode (SW or HW)
- *cb [in]*
 - Callback function address to be executed after an operation is finished
 - Send NULL pointer if you do not want Non-Blocking/IRQ mode active
- *cb_param [in]*
 - Opaque parameter to be passed when the callback is executed

Return Value:

- *Error code*

4.3.2 De-initialization

The Deinit API, as the name suggests, is used to de-initialize a Math Plug-in instance. This API completely de-initializes the driver and hardware associated to the plug-in structure.

```
extern int csl_math_plugin_unload(elp_math_descriptor* plugin);
```

Parameters:

- *plugin [in/out]*
 - Pointer to the plugin structure to be de-initialized

Return Value:

- *Error code.*

4.4 Updated Operations

This Section describes the changes performed in the existing DWC Cryptography Software Library APIs used for Public Key Cryptography.

4.4.1 RSA Sign

The RSA Sign API is used to perform a signature of a provided payload. More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the CRT Modular Exponentiation function. This is possible using the *rsaa_modexp_crt* function pointer present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
Int elprsa_sign_hash_ex(const uint8_t *in,          uint32_t inlen,
                      uint8_t *out,          uint32_t *outlen,
                      int padding,
                      elpprng_state *prngstate,
                      const elpprng_plugin *prng,
                      const elphash_plugin *hash, uint32_t saltlen,
                      elprsa_key *key)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet. If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.

4.4.2 RSA Verify

The RSA Verify API verifies a signature when provided both the signature and message digest (the original input to the signature generation function). More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the Modular Exponentiation function. This is possible using the *rsaa_modexp* function present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
int elprsa_verify_hash_ex( const uint8_t *sig,      uint32_t  siglen,
                          const uint8_t *hash,    uint32_t  hashlen,
                          int padding,
                          const elphash_plugin *phash,  uint32_t  saltlen,
                          int *stat,      elprsa_key  *key)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet. If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.

4.4.3 RSA Encrypt

The RSA Encrypt API performs encryption of a specified payload. More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the Modular Exponentiation function. This is possible using the *rsaa_modexp* function pointer present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
int elprsa_encrypt_key_ex(const uint8_t *in,      uint32_t  inlen,
                          uint8_t *out,      uint32_t  *outlen,
                          const uint8_t *lparam, uint32_t  lparamlen,
                          elpprng_state *prngstate, const elpprng_plugin *pr
ng
                          , const elphash_plugin *hash,
                          int padding, elprsa_key *key)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet. If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.

4.4.4 RSA Decrypt

The RSA Decrypt API performs decryption of a specified ciphertext. More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the CRT Modular Exponentiation function. This is possible using the *rsaa_modexp_crt* function pointer present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
int elprsa_decrypt_key_ex2(const uint8_t *in,      uint32_t inlen,
                          uint8_t *out,      uint32_t *outlen,
                          const uint8_t *lparam,  uint32_t lparamlen,
                          const elphash_plugin *hash,  int padding,
                          int *stat,      elprsa_key *key,
                          elpprng_state *prngstate, const elpprng_plugin *pr
ng)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet. If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.

4.4.5 ECC Sign

The ECC Sign API is used to perform a signature over a provided payload. More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the ECC Point Multiplication function. This is possible using the *ecca_ptmul* function pointer present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
int elpecc_sign_hash(const uint8_t *in,  uint32_t inlen,
                    uint8_t *out, uint32_t *outlen,
                    elpprng_state *prng,
                    const elpprng_plugin *pprng, elpecc_key *key)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet. If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.

4.4.6 ECC Verify

The ECC Verify API is used to verify a signature when provided both the signature and message digest (the original input to the signature generation function). More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the ECC Shamir's Trick function. This is possible using the *ecca_mul2add* function pointer present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
int elpecc_verify_hash(const uint8_t *sig,  uint32_t siglen,
                      const uint8_t *hash, uint32_t hashlen,
                      int *stat,      elpecc_key *key)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet.

If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.

4.4.7 ECC Encrypt

The ECC Encrypt API performs encryption of a specified payload. More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the ECC Point Multiplication function. This is possible using the *ecca_ptmul* function pointer present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
int elpecc_ecies_encrypt(
    const uint8_t *pt,          uint32_t ptlen,
    uint8_t **out,             uint32_t *outlen,
    uint32_t mackeylen,
    const elphash_plugin *hash,
    const elpprng_plugin *prng,    elpprng_state *prngstate,
    const uint8_t *shareddata1, uint32_t shareddata1len,
    const uint8_t *shareddata2, uint32_t shareddata2len,
    elpecc_key *recpkey)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet.

If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.

4.4.8 ECC Decrypt

The ECC Decrypt API performs decryption of a specified ciphertext. More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the ECC Point Multiplication function. This is possible using the *ecca_ptmul* function pointer present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
int elpecc_ecies_decrypt(
    const uint8_t *ct,          uint32_t ctlen,
    uint8_t **out,             uint32_t *outlen,
    uint32_t mackeylen,
    const elphash_plugin *hash,
    const elpprng_plugin *prng,    elpprng_state *prngstate,
    const uint8_t *shareddata1, uint32_t shareddata1len,
    const uint8_t *shareddata2, uint32_t shareddata2len,
    elpecc_key *recpkey,         int *stat)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet.

If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.

4.4.9 ECC Make Key

The ECC Make Key API is used to generate a new ECC private key. More detailed info about this API can be found in the DWC CSL User Guide Document.

With the Math Plug-in for PKA, this API redirects execution of its main core algorithm to PKA's implementation of the ECC Point Multiplication function. This is possible using the *ecca_ptmul* function pointer present in the Math Plug-in definition (see Section 5.1) which gets overloaded after the Plug-in initialization to point to PKA specific code.

```
int elpecc_make_key(elpprng_state *prng, const elpprng_plugin *pprng,  
                  int keysize, elpecc_key *key)
```

This API returns **ELPEOK** upon success, and **ELPEINPROGRESS** to indicate the operation did not finish yet.

If Non-Blocking mode is active and the returned value is **ELPEINPROGRESS**, the user must call the API once again with the same parameters in order to retrieve the result, only after the callback gets executed.